INGENIERÍA EN SOFTWARE

Ingeniería en Software

Apunte completo de estudio

Conceptos fundamentales · Patrones de diseño · Arquitecturas
Metodologías ágiles · SOLID · Calidad de software

Instituto Técnico Superior Córdoba

Materia: Ingeniería en Software

#	Sección
01	Software y Hardware · Definición, tipos y diferencias
02	Ciclo de vida del software (SDLC)
03	Calidad de software · ISO 25010 y deuda técnica
04	Patrones de diseño
05	Principios SOLID
06	Arquitecturas de software
07	Metodologías ágiles

01 · Software y Hardware

¿Qué es el software?

El software es el conjunto de instrucciones, programas, datos y documentación que le indica al hardware **qué hacer y cómo hacerlo**. Es intangible: no ocupa espacio físico pero sí espacio lógico (memoria, almacenamiento). Sin software, el hardware es solo un conjunto de piezas sin propósito.

Características distintivas del software

- **Intangible:** no se puede tocar ni ver. Existe como información procesable por una máquina.
- **No se desgasta:** puede copiarse infinitas veces sin degradarse físicamente, pero sí se vuelve obsoleto.
- **Se construye, no se fabrica:** cada proyecto es único. No existe línea de montaje: es un proceso creativo e intelectual.
- **Complejidad creciente:** a medida que crece, el mantenimiento se vuelve más costoso si no se aplica ingeniería desde el inicio.
- **Flexible pero frágil:** puede modificarse fácilmente, pero un pequeño cambio mal hecho puede romper todo el sistema.

Tipos de software

Tipo	Descripción	Ejemplos
Software de sistema	Gestiona el hardware y provee servicios base al resto del software.	Windows, Linux, macOS, drivers, BIOS
Software de aplicación	Resuelve necesidades concretas del usuario final.	Navegadores, apps móviles, ERP, CRM
Software de programación	Herramientas para crear otro software.	IDEs (VS Code), compiladores, depuradores
Firmware / Embedded	Software integrado en dispositivos físicos con recursos limitados.	Routers, microcontroladores, impresoras
Middleware	Intermediario entre aplicaciones y el sistema operativo.	Servidores de aplicaciones, brokers de mensajes

¿Qué es el hardware?

El hardware es el conjunto de componentes **físicos y tangibles** de un sistema informático. Sin software no hace nada útil; sin hardware el software no puede ejecutarse. Son interdependientes: uno sin el otro carece de utilidad.

- **CPU (Unidad Central de Proceso):** el "cerebro" del sistema. Ejecuta las instrucciones del software. Su velocidad se mide en GHz y su capacidad en núcleos.
- **RAM (Memoria principal):** almacenamiento temporal de los datos en uso. Volátil: se borra al apagar. Más RAM permite más procesos simultáneos.
- **Almacenamiento (HDD/SSD):** persistente: guarda datos aunque el sistema esté apagado. El SSD es mucho más rápido que el HDD mecánico.
- **GPU (Unidad de Procesamiento Gráfico):** procesa gráficos y operaciones paralelas masivas. Fundamental en inteligencia artificial y machine learning.
- **Placa madre (Motherboard):** conecta e intercomunica todos los componentes del sistema.
- **Periféricos:** dispositivos de entrada/salida: teclado, mouse, monitor, impresora.

Comparación: Hardware vs Software

Aspecto	Hardware	Software
Naturaleza	Físico y tangible	Lógico e intangible
Degradación	Se desgasta con el uso y el tiempo	No se desgasta, se vuelve obsoleto
Modificación	Se reemplaza físicamente	Se actualiza o refactoriza
Costo principal	Compra inicial	Desarrollo y mantenimiento continuo
Duplicación	Requiere fabricación	Se copia a costo casi cero
Visibilidad	Se puede ver y tocar	Solo se percibe a través de sus efectos

¿Qué es la Ingeniería en Software?

Es la disciplina que aplica principios de ingeniería al desarrollo, operación y mantenimiento de software de forma **sistemática, disciplinada y cuantificable**. No es solo programar: abarca todo el proceso desde los requerimientos hasta el retiro del sistema.

Programar	Ingeniería de Software
Escribir código que funciona	Construir sistemas de calidad, mantenibles y escalables
Foco en la solución inmediata	Foco en el ciclo de vida completo del software
Individual / técnico	Trabajo en equipo, procesos, gestión y calidad
El código "corre"	El código es correcto, eficiente, testeable y documentado

> **Tip:** "Que el código funcione" es apenas el punto de partida. La ingeniería empieza cuando ese código tiene que seguir funcionando, creciendo y siendo mantenido por un equipo durante años.

02 · Ciclo de Vida del Software (SDLC)

El **SDLC** (Software Development Life Cycle) es el proceso estructurado para planificar, crear, probar y entregar software. Define las fases que atraviesa un proyecto desde la idea inicial hasta el retiro del sistema.

Las 7 fases del SDLC

- **1 · Planificación.** Se define el alcance, viabilidad económica y técnica, cronograma y asignación de recursos. *Producto:* plan de proyecto, análisis costo-beneficio.
- **2 · Análisis de Requerimientos.** Se capturan las necesidades del cliente. Se distinguen requerimientos funcionales (qué hace el sistema) y no funcionales (rendimiento, seguridad, usabilidad). *Producto:* SRS (Software Requirements Specification).
- **3 · Diseño.** Se define la arquitectura, patrones a usar, base de datos, interfaces e integraciones. Incluye diseño de alto nivel (arquitectura) y bajo nivel (clases, módulos). *Producto:* documento de diseño, diagramas UML.
- **4 · Implementación.** Los desarrolladores escriben el código siguiendo el diseño. Se aplican estándares, control de versiones y revisiones de código (code review).
- **5 · Pruebas (Testing).** Verificación (¿construimos el sistema correctamente?) y validación (¿construimos el sistema correcto?). Pruebas unitarias, integración, sistema y aceptación.
- **6 · Despliegue (Deployment).** El software se entrega al entorno de producción. Puede ser gradual (canary release, blue-green) o completo.
- **7 · Mantenimiento.** Correctivo (bugs), adaptativo (cambios del entorno), perfectivo (mejoras) y preventivo (refactoring). Es la fase más larga y costosa del ciclo.



Las 7 fases del ciclo de vida del software.

Modelos de proceso

Cascada (Waterfall)

Secuencial y estricto. Cada fase debe terminar completamente antes de iniciar la siguiente. Documentación exhaustiva al final de cada etapa.

Ventajas	Desventajas
Fácil de gestionar y documentar	Rígido ante cambios de requisitos

Ventajas	Desventajas
Claro para proyectos con requisitos estables	El cliente ve el producto recién al final
Ideal para sistemas críticos bien definidos	Errores detectados tarde son muy costosos

Iterativo / Incremental

Se construye el sistema en ciclos cortos, entregando partes funcionales en cada iteración. Cada ciclo agrega nueva funcionalidad.

Ventajas	Desventajas
Feedback temprano del cliente	Requiere más coordinación y planificación
Riesgo reducido: errores se detectan antes	El alcance puede crecer descontroladamente
Se adapta bien a requisitos cambiantes	Necesita cliente disponible durante el proceso

Espiral

Combina iteración con **análisis de riesgo** en cada vuelta. Cada ciclo incluye planificación, análisis de riesgo, ingeniería y evaluación.

Ventajas	Desventajas
Gestión activa del riesgo	Costoso: requiere experiencia en gestión de riesgos
Flexible y adaptable	Difícil de escalar en proyectos pequeños
Bueno para proyectos grandes y complejos	

Requerimientos funcionales vs no funcionales

Funcionales — qué hace el sistema	No funcionales — cómo lo hace
El sistema permite registrar usuarios con email y contraseña.	El login debe responder en menos de 200 ms.
El usuario puede filtrar productos por precio y categoría.	El sistema soporta 10.000 usuarios concurrentes.
Se envía un email de confirmación al completar una compra.	Las contraseñas deben almacenarse hasheadas (bcrypt).
El administrador puede generar reportes en PDF.	Disponibilidad del 99.9% mensual.

03 · Calidad de Software

Un sistema puede funcionar y aun así tener mala calidad. La IS busca producir software **correcto, confiable, eficiente, mantenible y usable**, no solo que "corra".

Atributos de calidad — ISO 25010

- **Funcionalidad:** el sistema hace lo que se especificó que debía hacer.
- **Confiabilidad:** es estable bajo condiciones normales y anómalas. Se recupera de errores.
- **Usabilidad:** el usuario puede aprender y usar el sistema eficientemente, con satisfacción.
- **Eficiencia:** usa los recursos (CPU, memoria, red) de forma proporcional a la carga.
- **Mantenibilidad:** es fácil modificar, corregir y extender. El código es legible y modular.
- **Portabilidad:** puede ejecutarse en distintos entornos con cambios mínimos.
- **Seguridad:** protege los datos y el acceso no autorizado. Resiste ataques.
- **Compatibilidad:** puede coexistir e integrarse con otros sistemas.

Deuda técnica

Es el **costo futuro** de haber tomado atajos en el desarrollo. Como la deuda financiera, genera "intereses": cada cambio futuro es más lento y costoso. Se acumula cuando se prioriza la velocidad sobre la calidad del diseño.

- Código duplicado en lugar de abstraer en funciones o clases.
- Funciones de 500 líneas que hacen múltiples cosas a la vez.
- Variables con nombres sin significado: `x`, `temp`, `data`, `aux`.
- Sin pruebas automatizadas: cualquier cambio puede romper algo sin que se note.
- Sin documentación ni comentarios en lógica compleja.
- Dependencias desactualizadas con vulnerabilidades conocidas.

> **Ojo:** La IS propone pagar la deuda técnica con **refactoring**: reestructurar el código sin cambiar su comportamiento externo. El refactoring no agrega funcionalidad, mejora la estructura interna del código para que sea más fácil de mantener.

Cohesión y acoplamiento

Son las dos métricas de diseño más importantes en IS. El objetivo siempre es **alta cohesión y bajo acoplamiento**.

Cohesión	Acoplamiento
Grado en que los elementos de un módulo pertenecen juntos y tienen una responsabilidad única y bien definida.	Grado de dependencia entre módulos. A menor acoplamiento, más fácil es cambiar uno sin afectar al otro.
Alta cohesión: cada módulo hace una sola cosa bien.	**Bajo acoplamiento:** los módulos se pueden cambiar independientemente.

04 · Patrones de Diseño

Los patrones de diseño son **soluciones reutilizables a problemas recurrentes** en el diseño de software. No son código que se copia: son plantillas conceptuales que se adaptan al contexto. Fueron popularizados por el libro *"Gang of Four"* (GoF) de 1994.

Patrones creacionales — cómo se crean los objetos

Controlan el proceso de creación de objetos, haciéndolo más flexible y desacoplado del resto del sistema.

Singleton

Garantiza que una clase tenga una **única instancia** en toda la aplicación y provee un punto de acceso global a ella. Útil para configuraciones globales, loggers o conexiones a base de datos.

Singleton

```
public class Configuracion {
    private static Configuracion _instancia;
    private Configuracion() { } // constructor privado

    public static Configuracion ObtenerInstancia() {
        if (_instancia == null)
            _instancia = new Configuracion();
        return _instancia;
    }
}
```

Factory Method

Define una interfaz para crear objetos, pero deja que las subclases decidan qué clase instanciar. Desacopla la creación de objetos de su uso.

Factory Method

```
public abstract class NotificadorFactory {
    public abstract INotificador Crear();
}

public class EmailFactory : NotificadorFactory {
    public override INotificador Crear() => new EmailNotificador();
}
```

Builder

Construye objetos complejos **paso a paso**. Permite producir distintos tipos de objetos usando el mismo código de construcción. Ideal cuando un objeto tiene muchos parámetros opcionales.

Builder

```
var usuario = new UsuarioBuilder()
    .ConNombre("Ana")
    .ConEmail("ana@mail.com")
    .ConRol("admin")
    .Build();
```

Patrones estructurales — cómo se componen las clases

Definen cómo se organizan y relacionan las clases para formar estructuras más grandes y flexibles.

Adapter

Permite que interfaces incompatibles trabajen juntas. Como un adaptador de enchufe: convierte una interfaz en otra que el cliente espera. Muy usado para integrar librerías externas sin modificar el código existente.

Adapter

```
// La API externa tiene una interfaz incompatible
public class ApiExterna { public string FetchData() { ... } }

// El adapter la adapta a nuestra interfaz
public class ApiAdapter : IDataSource {
    private ApiExterna _api = new ApiExterna();
    public string ObtenerDatos() => _api.FetchData();
}
```

Decorator

Agrega responsabilidades a un objeto de forma **dinámica**, sin modificar su clase original. Es una alternativa flexible a la herencia. Permite añadir logging, caché o validaciones sin tocar el código original.

Decorator

```
public class LoggingService : IProductoService {
    private readonly IProductoService _inner;
    public LoggingService(IProductoService inner) => _inner = inner;

    public async Task ObtenerAsync(int id) {
        Console.WriteLine($"Buscando producto {id}...");
        var result = await _inner.ObtenerAsync(id);
        Console.WriteLine($"Encontrado: {result.Nombre}");
        return result;
    }
}
```

Facade

Provee una interfaz **simplificada** a un subsistema complejo. Oculta la complejidad interna y expone solo lo que el cliente necesita. Muy usado en capas de servicios.

Facade

```
// En lugar de coordinar 5 servicios manualmente...
public class PedidoFacade {
    public async Task ProcesarPedido(Pedido p) {
        await _stock.ReservarAsync(p);
        await _pago.CobrarAsync(p);
        await _envio.ProgramarAsync(p);
        await _notif.EnviarConfirmacionAsync(p);
    }
}
```

Patrones de comportamiento — cómo se comunican los objetos

Definen cómo los objetos interactúan y se reparten responsabilidades entre sí.

Observer

Define una relación **uno-a-muchos**: cuando un objeto cambia de estado, todos sus dependientes son notificados automáticamente. Es la base de los sistemas de eventos y notificaciones.

Observer

```
public interface IObservador { void Actualizar(string evento); }

public class SistemaNoticias {
    private List<IObservador> _suscriptores = new();
    public void Suscribir(IObservador o) => _suscriptores.Add(o);
    public void Publicar(string noticia) {
        foreach (var s in _suscriptores) s.Actualizar(noticia);
    }
}
```

Strategy

Define una familia de **algoritmos intercambiables**. El cliente elige qué algoritmo usar en tiempo de ejecución sin cambiar el código del contexto.

Strategy

```
public interface IOrdenamiento { List<T> Ordenar(List<T> l); }
public class BubbleSort : IOrdenamiento { /* ... */ }
public class QuickSort : IOrdenamiento { /* ... */ }

// El contexto usa cualquier estrategia sin saber cuál es
var ordenador = new Ordenador(new QuickSort());
ordenador.Ejecutar(miLista); // cambiable en runtime
```

Repository

Abstrae el acceso a datos detrás de una interfaz. El resto del sistema no sabe si los datos vienen de una base de datos SQL, un archivo, una API o memoria. Facilita el testing con mocks.

Repository

```
public interface IProductoRepo {
    Task<List<Producto>> ObtenerTodosAsync();
    Task<Producto> ObtenerPorIdAsync(int id);
    Task GuardarAsync(Producto p);
}
// Implementaciones: SqlProductoRepo, MockProductoRepo...
```

Command

Encapsula una solicitud como un objeto. Permite deshacer operaciones, encolar comandos y registrar un historial de acciones. Muy usado en editores de texto y sistemas de workflows.

05 · Principios SOLID

SOLID es un acrónimo de 5 principios de diseño orientado a objetos formulados por Robert C. Martin. Aplicarlos reduce el acoplamiento, aumenta la cohesión y hace el código más fácil de entender, probar y cambiar sin romper nada.

S — Single Responsibility (Responsabilidad única)

Una clase debe tener **una sola razón para cambiar**. Si una clase gestiona tanto la lógica de negocio como el acceso a la base de datos, cualquier cambio en la BD afecta la lógica. Cada clase debe hacer una sola cosa y hacerla bien.

MAL — una clase con múltiples responsabilidades

```
public class Usuario {
    public void Guardar() { /* accede a la BD */ }
    public void EnviarEmail() { /* envía correo */ }
    public void GenerarReporte() { /* crea PDF */ }
}
```

BIEN — responsabilidades separadas

```
public class UsuarioRepository { public void Guardar() {} }
public class EmailService     { public void Enviar() {} }
public class ReporteService   { public void Generar() {} }
```

O — Open/Closed (Abierto/Cerrado)

Las entidades de software deben estar **abiertas para extensión pero cerradas para modificación**. Podés agregar nueva funcionalidad sin tocar el código existente. Se logra mediante interfaces y herencia.

BIEN — extensible sin modificar

```
// En lugar de modificar la clase para cada nuevo descuento:
public interface IDescuento { decimal Aplicar(decimal precio); }
public class DescuentoVIP        : IDescuento { /* 20% */ }
public class DescuentoEstudiante : IDescuento { /* 15% */ }
// Nuevo descuento = nueva clase, sin tocar las existentes
```

L — Liskov Substitution (Sustitución de Liskov)

Los objetos de una subclase deben poder **sustituir** a los de la clase padre sin romper el programa. Si tenés un método que espera un `Animal`, un `Perro` debe funcionar perfectamente en su lugar sin comportamientos inesperados.

> **Ojo:** Un `Pinguino` que hereda de `Ave` con `Volar()` pero no puede volar (lanza excepción) **viola el contrato** de la clase padre. La herencia debe respetar el comportamiento esperado.

I — Interface Segregation (Segregación de Interfaces)

Es mejor tener **muchas interfaces pequeñas y específicas** que una grande y general. Ninguna clase debería verse obligada a implementar métodos que no usa.

MAL — interfaz demasiado grande

```
interface IAnimal { void Volar(); void Nadar(); void Correr(); }
```

BIEN — interfaces específicas y componibles

```
interface IVolador  { void Volar(); }
interface INadador  { void Nadar(); }
interface ICorredor { void Correr(); }

public class Pato  : IVolador, INadador  { /* implementa las dos */ }
public class Perro : INadador, ICorredor { /* solo las que usa */ }
```

D — Dependency Inversion (Inversión de Dependencias)

Los módulos de alto nivel no deben depender de los de bajo nivel. Ambos deben depender de **abstracciones** (interfaces). Las implementaciones concretas no deben ser conocidas por el código que las usa. Esto hace el código testeable y flexible.

MAL — el servicio crea su propia dependencia concreta

```
public class PedidoService {
    private SqlPedidoRepo _repo = new SqlPedidoRepo(); // acoplado
}
```

BIEN — depende de la abstracción, recibida por constructor

```
public class PedidoService {
    private readonly IPedidoRepo _repo;
    public PedidoService(IPedidoRepo repo) => _repo = repo;
    // Funciona con SqlRepo, MockRepo, o cualquier implementación
}
```

06 · Arquitecturas de Software

La arquitectura de software define la **estructura de alto nivel** de un sistema: cómo se organizan sus componentes, cómo se comunican entre sí y qué decisiones técnicas fundamentales guían su construcción.

Monolito

Toda la aplicación se desarrolla y despliega como una **sola unidad**. El frontend, backend y acceso a datos están integrados en un único proyecto desplegable.

Estructura típica de un monolito

```
MiApp/
├─ Controllers/    // endpoints HTTP
├─ Services/       // lógica de negocio
├─ Repositories/   // acceso a datos
├─ Models/         // entidades
└─ Program.cs      // punto de entrada único
```

Ventajas	Desventajas
Simple de desarrollar al inicio	Escala toda la app o ninguna parte
Un solo deployment	Un bug grave puede tirar todo el sistema
Sin latencia entre componentes	Tech stack único y rígido
Debugging directo y sencillo	Los equipos se pisan entre sí al crecer

Ideal para: MVPs, proyectos nuevos, equipos pequeños (menos de 10 personas).

Microservicios

La aplicación se divide en **servicios pequeños e independientes**, cada uno con su propia base de datos, responsabilidad, tecnología y ciclo de despliegue.

Estructura de microservicios

```
ServicioUsuarios/   // su propia BD, su propio deploy
ServicioProductos/  // escala de forma independiente
ServicioPedidos/    // puede estar en otro lenguaje
ServicioNotif/      // se comunican por API o mensajes async
```

Ventajas	Desventajas
Escalado independiente por servicio	Alta complejidad operativa (K8s, CI/CD)

Ventajas	Desventajas
Fallo aislado: un servicio caído no tira todo	Latencia de red entre servicios
Equipos autónomos por servicio	Testing distribuido es complejo
Distintas tecnologías por servicio	Necesita infraestructura DevOps madura

Ideal para: sistemas de alta escala, muchos equipos, alta disponibilidad (Netflix, Amazon, Uber).

Otras arquitecturas importantes

- **Capas (N-tier):** organiza el sistema en capas horizontales: Presentación → Lógica de negocio → Acceso a datos → Base de datos. Cada capa solo interactúa con la adyacente.
- **Hexagonal (Ports & Adapters):** el núcleo de negocio no depende de frameworks ni de la BD. Todo el exterior (HTTP, BD, email) es un adaptador intercambiable. Facilita el testing.
- **Orientada a eventos (Event-driven):** los componentes se comunican publicando y consumiendo eventos asincrónicos. Muy usada en microservicios. Herramientas: Kafka, RabbitMQ, Azure Service Bus.
- **Serverless / FaaS:** funciones individuales que se ejecutan en respuesta a eventos. El proveedor cloud gestiona la infraestructura. Pago por uso. Ej: AWS Lambda, Azure Functions.

¿Cuándo usar Monolito vs Microservicios?

Usá Monolito si...	Usá Microservicios si...
El proyecto es nuevo o pequeño	El sistema necesita escalar masivamente
El equipo tiene menos de 10 personas	Hay múltiples equipos autónomos
Querés iterar rápido y validar ideas	Distintas partes tienen requerimientos muy diferentes
No tenés infraestructura DevOps madura	Ya tenés CI/CD, contenedores y experiencia

> **Regla práctica:** empezá con un monolito bien estructurado. Extraé microservicios solo cuando el dolor sea real, no como decisión prematura.

07 · Metodologías Ágiles

Las metodologías ágiles son marcos de trabajo **iterativos e incrementales** para gestionar proyectos de software. Nacieron como respuesta a los problemas del modelo en cascada: requisitos que cambian, entregas tardías y poca colaboración con el cliente.

El Manifiesto Ágil (2001) — 4 valores

17 desarrolladores firmaron el Manifiesto Ágil estableciendo 4 valores y 12 principios que guían el desarrollo ágil. Los ítems de la izquierda tienen más valor que los de la derecha, aunque los de la derecha también importan.

Más valor	sobre	Menos valor
Individuos e interacciones	sobre	procesos y herramientas
Software funcionando	sobre	documentación exhaustiva
Colaboración con el cliente	sobre	negociación contractual
Respuesta al cambio	sobre	seguir un plan

Scrum

Framework iterativo basado en **sprints** de 1 a 4 semanas. Al final de cada sprint se entrega un incremento funcional y potencialmente desplegable.

Roles	Artefactos	Ceremonias
Product Owner: prioriza el backlog.	**Product Backlog:** lista de todo lo que puede hacerse.	**Sprint Planning:** qué se va a hacer.
Scrum Master: facilita y elimina obstáculos.	**Sprint Backlog:** tareas del sprint actual.	**Daily Standup:** sincronización diaria (15 min).
Dev Team: diseña, construye y prueba.	**Incremento:** producto entregable al final de cada sprint.	**Sprint Review:** demo al cliente. **Retrospectiva:** cómo mejorar el proceso.

Kanban

Sistema de **flujo continuo** basado en un tablero visual con columnas. No tiene sprints fijos ni roles formales. El foco es limitar el trabajo en progreso (WIP) para evitar cuellos de botella y mantener un flujo predecible.

Por hacer	En progreso	En revisión	Hecho
Backlog de tareas pendientes que aún no se iniciaron.	Tareas activas. WIP limitado para evitar multitasking.	Tareas terminadas esperando revisión o testing.	Completadas, revisadas y entregadas.

Scrum vs Kanban — cuándo usar cada uno

	Scrum	Kanban
Estructura	Sprints fijos de 1-4 semanas	Flujo continuo sin iteraciones
Roles	PO, SM, Dev Team definidos	Sin roles obligatorios
Entregas	Al final de cada sprint	Continuas cuando están listas
Mejor para	Proyectos con sprints definidos, equipo estable	Soporte, mantenimiento, ops, flujo impredecible
Cambios en curso	Fuera del sprint (al siguiente)	Se pueden incorporar en cualquier momento

08 · Conceptos Complementarios

Control de versiones (Git)

Sistema que registra la historia del código. Permite trabajar en equipo sin pisar cambios, revertir errores, crear ramas de desarrollo paralelas y hacer code review antes de integrar cambios al código principal.

Comandos básicos de Git

```
git init                        # iniciar repositorio local
git branch feature/login        # nueva rama para una funcionalidad
git add . && git commit -m "feat: agregar login"
git merge feature/login         # integrar al main
git revert HEAD                 # deshacer el último commit
```

Testing — Pirámide de pruebas

Las pruebas se organizan en una pirámide: muchas pruebas unitarias (rápidas y baratas), menos pruebas de integración, y aún menos pruebas E2E (lentas y costosas). El objetivo es detectar errores lo más temprano posible en el proceso.



La base de la pirámide concentra la mayor cantidad de pruebas.

DevOps y CI/CD

DevOps une desarrollo (Dev) y operaciones (Ops) en un flujo continuo. **CI** (Continuous Integration): cada push dispara tests automáticos. **CD** (Continuous Delivery/Deployment): si los tests pasan, el código se despliega automáticamente a staging o producción.

Principios de código limpio: DRY, KISS, YAGNI

- **DRY** (Don't Repeat Yourself): abstraer la lógica duplicada.
- **KISS** (Keep It Simple): la solución más simple es la mejor.
- **YAGNI** (You Aren't Gonna Need It): no codificar lo que no se necesita hoy.

UML — Diagramas de diseño

El Lenguaje de Modelado Unificado es el estándar para visualizar el diseño de sistemas de software. Los más usados en IS: diagrama de clases, casos de uso, secuencia, estado y actividad. Permiten comunicar decisiones de diseño sin ambigüedad.

Tipos de pruebas de software

Tipo	Qué prueba	Velocidad	Quién la escribe
Unitaria	Una función o clase aislada	Muy rápida	El desarrollador
Integración	Interacción entre componentes	Media	QA / Dev
Sistema	El sistema completo end-to-end	Lenta	Cliente / QA
Aceptación (UAT)	Que el sistema cumple los requisitos del cliente	Variable	Cliente / QA
Regresión	Que los cambios no rompieron lo que funcionaba	Variable	Automatizada

Glosario rápido

Término	Definición
Requerimiento	Condición o capacidad que el sistema debe cumplir.
Refactoring	Mejorar la estructura interna del código sin cambiar su comportamiento externo.
Deuda técnica	Costo futuro de haber tomado atajos de diseño en el desarrollo.
Abstracción	Ocultar la complejidad interna y exponer solo lo esencial al exterior.
Encapsulamiento	Agrupar datos y comportamiento en una clase, ocultando el estado interno.
Herencia	Una clase reutiliza atributos y métodos de otra clase padre.
Polimorfismo	Un mismo método se comporta de forma distinta según el objeto que lo ejecuta.
Acoplamiento	Grado de dependencia entre módulos. Buscamos bajo acoplamiento.
Cohesión	Grado en que los elementos de un módulo pertenecen juntos. Buscamos alta cohesión.
Escalabilidad	Capacidad del sistema de manejar mayor carga sin perder rendimiento.
Disponibilidad	Porcentaje de tiempo que el sistema está operativo y accesible.
Deploy	Proceso de publicar una versión del software en un entorno (staging, producción).